CAMPUSSHARE X

Project Report

The trust layer for shared resources

A full-stack campus resource-exchange application

Project-aware discovery • Eligibility checks • Fair allocation • Condition-recorded handover • Auditable history

Selected setting	University campus resource exchange
Primary use case	Equipment sharing for student projects and research
Application stack	React · TypeScript · Express · SQLite
Data status	Fictional starter catalogue; replace with authorised inventory
Document status	Prepared for team review and hackathon submission

Important submission note

The supplied hackathon statement requires development during the official 48-hour build window and disclosure of AI/tool assistance. The team must confirm whether this generated source can be used as a starting point, preserve accurate development history and disclose assistance honestly.

1. Executive summary

CampusShare X is a resource-sharing application for universities and similar institutions. It connects discovery to eligibility checks, owner-controlled approvals, scheduling conflict prevention, fair-use controls, condition-recorded handovers, return extensions, incident review and an audit trail. Its main differentiators are SmartMatch, FairShare and the Resource Passport.

SmartMatch helps users search by purpose rather than exact asset name. TrustGuard checks account verification and resource-specific training prerequisites. FairShare applies an active-reservation limit and transparent waitlisting. The Resource Passport keeps resource-specific records of handover, return and related events. All access and penalty decisions remain under authorised human review.

2. Problem and selected use case

Colleges, laboratories, makerspaces and community organisations may own valuable resources that sit unused for long periods, while students and researchers cannot easily afford to purchase equipment or know how to access it. A basic booking calendar does not resolve eligibility, safety, competing demand, condition documentation, owner approval, late return or incident investigation.

CampusShare X focuses on the campus setting and a practical transaction: a student needs equipment for an IoT prototype. They discover the ESP32 kit and sensors, request a suitable time, pass server-side eligibility checks, obtain an owner decision, document the initial condition, check the kit out, return it and record the final condition. If there is a discrepancy, an authorised manager reviews it.

Evidence basis: The supplied hackathon problem statement identifies unused institutional resources, access barriers, eligibility requirements, scheduling conflicts, property condition, fair allocation and accountability as the problem areas. No independent campus utilisation survey was included with this project; institution-specific impact claims require future research and measurement.

3. Target users

- **Requester:** Student, researcher or project team member who discovers a resource, reviews requirements and submits a booking request.
- **Resource owner:** Person authorised to list and manage assets, approve/decline requests, plan unavailable periods, record handovers and review incidents.
- **Administrator / institutional manager:** Reviews account verification, assigns owner roles, verifies training records and inspects authorised institutional activity.

4. Product objectives

1. Discover equipment and bookable spaces using category, location and capability tags.
2. Explain why a resource matches a project goal.
3. Enforce verified profile and required training before a booking proceeds.
4. Let an owner approve, reject or waitlist requests with recorded reasons.
5. Detect conflicts and present current confirmed bookings, pending demand and scheduled blocks for the selected date.
6. Record initial and final condition at handover/return.
7. Handle extension requests, cancellations, overdue reminders and incidents.
8. Preserve a traceable history of important decisions and actions.
9. Offer transparent usage limits and queue ordering to discourage monopolisation.
10. Provide resource-use summaries derived from stored transactions.

5. Features and implementation status

5.1 Authentication and roles

The application uses account registration/login, bcrypt password hashing and HTTP-only session cookies. Newly registered students are pending until reviewed. An administrator may verify an account and assign a verified user the resource-owner role. Server routes check role and resource ownership; the client role display is not an authorization boundary.

5.2 Resource catalogue and Passport

A resource record contains name, category, description, location, owner, capability tags, required training identifiers, operational status, asset code and use/return instructions. Equipment and spaces use the same resource model. Resource Passport views show identity, requirements and related audit activity.

The initial catalogue contains fictional starter records for local validation. These are not verified real campus inventory, availability or institutional policy.

5.3 SmartMatch

The user enters a natural-language project goal. The server derives candidate tags using deterministic aliases; if an LLM API key is configured, the LLM may extract intent but is restricted to an allowlist of capability tags present in the catalogue. Match scores rank functional relevance and provide reasons. A score is not an availability probability or permission decision. The application shows schedule data from the database separately and checks again on booking submission.

5.4 TrustGuard / eligibility

The server verifies profile status and every resource's required training. The same eligibility policy is checked at booking creation, owner approval and check-in. Users cannot grant their own role or training status from the client. An administrator-controlled training endpoint records the verifier and creates an audit event. Institution-specific membership, deposit and supervised-use rules may need additional fields/integrations for a particular campus.

5.5 Booking and schedule

A request stores resource, user, date, times, purpose, status and decision reason. The server validates dates, future start time, duration, active reservation count, duplicate requests, confirmed conflicts and maintenance blocks. The owner can approve, reject or waitlist a request. When an overlapping pending request loses a conflict to an approval, it is moved to the waitlist with an explanation. The management interface displays pending/waitlisted requests oldest-first; owners make final decisions.

5.6 FairShare

FairShare is intentionally rule-based and explainable. It limits a user to three active/upcoming reservations at a time, prevents overlapping confirmed booking approval, captures waiting-list status/reason and shows queue requests in request-time order. It is not a machine-learning allocation model and does not claim a measured fairness improvement. Institution policy can later define priority tiers and audit any authorised override.

5.7 Check-in, return and condition reports

Check-in changes an approved booking to checked out and stores a before condition report. Return stores an after report, updates the resource-use counter and closes the booking. Non-good return conditions automatically open an incident for review; the app does not automatically assign blame or impose a penalty. Reports are text-based in the current version; photo uploads are future work.

5.8 Extensions and scheduling blocks

A requester may ask to extend a checked-out booking. The owner or administrator can approve/reject the extension. Before approval, the server checks booking state, the new end time, duration and overlapping bookings/blocks. Owners can record a future unavailable/maintenance interval; new requests and approvals are checked against it.

5.9 Notifications, incidents and audit

In-app notifications cover booking decisions, user verification, extension decisions, incident status, upcoming bookings and overdue returns. They are stored in the database; outbound email/WhatsApp/SMS is not connected. An incident has a reporter, description, severity, status, reviewer and resolution. The audit list records sign-in, registration, resource listing/status, booking decisions, handover, return, extension, verification, training and incident actions. CSV export is permission-scoped to the current workspace.

5.10 Analytics

The dashboard presents resource counts, recorded return/use counts, booking lifecycle status, pending decisions and open incidents. These are operational summaries, not causal claims. Seeded data must be replaced before institutional conclusions are drawn.

6. End-to-end workflow

1. Owner/manager creates a resource and defines capability tags, location, instructions and required training.
2. Requester signs in and the system loads catalogue records from the server.
3. Requester searches or enters a project description; SmartMatch ranks suitable resources.
4. The requester checks policy and date-specific confirmed bookings, pending requests and time blocks.
5. The requester submits a booking request; the server repeats all eligibility and scheduling validations.
6. Owner reviews the request and approves, rejects or places it on a waitlist. Decisions and reasons are recorded.
7. An authorised user records check-in and initial condition.
8. The requester returns the resource and records final condition.
9. A condition discrepancy or incident is sent for human review; extension/cancellation/overdue events are captured as appropriate.
10. Both requester and authorised owner can view the relevant transaction history; the institution can export authorised audit records.

7. System architecture and technology

- **Frontend:** React, TypeScript, Vite, CSS and Lucide icons.
- **API/backend:** Node.js, Express.
- **Database:** SQLite via better-sqlite3, with foreign keys, indexes and WAL journaling enabled.
- **Authentication:** bcrypt password hashes, opaque random session token stored as a hash, HTTP-only SameSite cookie.
- **Hardening:** Helmet security headers, authentication rate limit, JSON/body limits, parameterized SQL, owner/role guards, server-side validation, same-origin mutation check in production.
- **AI:** Optional OpenAI-compatible chat-completions API for intent-to-tag extraction; deterministic fallback if unavailable.
- **Deployment shape:** One Node service serves the built web client and API. SQLite requires persistent storage and a single application instance. A larger deployment should migrate to managed PostgreSQL and review transactional conflict constraints.

8. Data model

Entity	Role
users	Identity, role, profile verification and password hash
sessions	Hashed session token and expiry
resources	Shared item/space, owner, location, policy tags and state
training_records	Verified/expired training by user and training identifier
bookings	Requested schedule, state, purpose, decisions and before/after summary
condition_reports	Stage-specific inspection record and notes
resource_blocks	Owner-defined future unavailability/maintenance intervals
booking_extensions	Requested/approved/rejected return-time changes
incidents	Issue report, severity, resolution and reviewer
notifications	In-app events, read status and timestamp
audit_events	Actor, action, target, reason/detail and linked resource/booking

9. Privacy, safety and fairness

- Use fictional or anonymized demonstration accounts and resources unless authorised.
- Do not expose password hashes, session tokens or private training documents through the API.
- Store only the information required for resource sharing; establish institution-specific retention and backup rules before real use.
- Keep private files private if photo/document storage is added.
- Do not allow AI to bypass eligibility or approve access. The current model only suggests capability tags.
- Have an authorised human review damage reports, access exceptions and penalties.
- Publish fairness/queue rules, enforce reservation limits consistently and audit owner overrides.
- Treat an initial/final condition difference as an issue to review, not automatic proof of fault.

10. Testing

The automated API test file covers health/authentication, restricted training, competing requests and waitlisting, owner approval, check-in, approved extension, return condition and incident resolution, training verification, role boundaries and audit CSV. The test must be run with installed dependencies using npm test. The manual plan in docs/TEST_PLAN.md covers the UI and deployment.

Verification note: This package was assembled in an environment that could not reach the npm registry. The server JavaScript and test file were syntax-checked here, and the TSX was syntax-transpiled; a dependency-based npm install, full TypeScript build and integration test execution still need to be run in a network-enabled environment. Record real results before submission.

11. Limitations

- Starter records are fictional and are not an authorised institutional catalogue.
- Outbound notification channels and identity-provider integrations are not connected.
- Condition reporting is text-only; photo attachments and image comparisons are not implemented.
- Eligibility currently models institutional verification and required training identifiers; deposits, formal memberships and staff-supervision schedules may need additional policy data.

- SQLite is single-instance storage; no multi-region or horizontally scaled deployment is provided.
- FairShare uses transparent rule-based controls; no measured outcome claims or machine-learning fairness optimisation are included.
- The application has not been penetration-tested or approved for real restricted equipment or sensitive institutional data.

12. Future work

1. Add managed PostgreSQL and transactional range/exclusion constraints for multi-instance hosting.
2. Integrate institution identity and trusted training sources.
3. Add secure condition photo upload with size/type limits and access-controlled storage.
4. Add outbound email/WhatsApp notifications through institution-approved providers.
5. Improve recurring scheduling, maintenance management and automatic waitlist promotion under published policy.
6. Evaluate allocation rules using simulated competing requests and report measured fairness/access metrics.
7. Add demand forecasting and predictive maintenance only after enough valid usage data exists.

13. Event and AI disclosure

The supplied hackathon statement requires the submitted product to be developed during its official 48-hour window and requires disclosure of significant AI/tool use and pre-existing components. The team must confirm whether and how this generated source may be used under the event rules, disclose AI assistance honestly, and never claim that work was completed during the official window if it was not. See docs/TEAM_AND_SUBMISSION.md before submitting.